

Running a specialist on a story by hand

This is the manual fallback, not the normal path. Normally a developer moves a story to [In Progress](#) and the app does the rest: it checks the story's dependencies, creates the story branch, runs the specialist in its sandbox, requests the mover as the PR's reviewer, and watches the PR.

Use this document when you need to run the specialist yourself, in Claude Code, against a local checkout. That happens when you are debugging a dispatch, when the app is not deployed, or when the target is something the app cannot reach yet: a repo hosted anywhere other than GitHub, or work that needs more than one repo checked out.

You do three things: set the branch up, paste a prompt, review what comes back.

Before you start: what running by hand gives up

Do not move the story to [In Progress](#). That status move is what triggers the app. If you run the specialist by hand and also move the story, the app dispatches a second specialist at the same story. Leave it in To-Do while you work, and move it yourself once the PR is merged or the run is abandoned.

Running by hand skips everything the app does around the specialist:

- **No reviewer is requested.** Ask a developer other than yourself to review the PR.
 - **No revision rounds.** When a reviewer asks for changes on an app-dispatched PR, the app sends the specialist back to address them, up to three rounds. By hand, nothing does that. Run the specialist again yourself (see "Answering a review" below), or make the changes directly.
 - **No automatic move back to To-Do.** When an app-dispatched run stops without a PR, the app moves the story back. By hand, the board shows whatever you leave it at.
 - **No dependency check or branch creation.** The specialist still checks its blocking dependencies and verifies the branch chain itself, but you create the story branch.
-

First time only

1. Claude Code, from Claude Desktop, pointed at the workspace folder

Start a Claude Code session in Claude Desktop and point it at the **workspace folder**, the parent directory holding all the repos, not at a single repo. The specialist needs to read files, run git, run the tests, and open a pull request, and it needs more than one repo in view to do it.

2. Lay the workspace out

~/work/<project>/	← point Claude Code here
├─ <surface-a>/	← a product repo (frontend, backend, ...)
├─ <surface-b>/	← another product repo
└─ intent-to-production/	← the framework repo (agent definition, skills)

Everything in one place because reading across repos matters: a frontend story has to check the API the backend actually shipped, not the one the story describes.

Writes go to one repo only. A story can carry more than one `surface:` label, but they always resolve to the same repo and ref, so there is one branch and one PR. The surface registry (the project's `Surfaces` document) says which repo that is and which directory each surface occupies inside it. The other repos are read-only.

3. Clone `intent-to-production`

The specialist definition (`agents/specialist.md`) and its skills live there, not in the product repos. Note the absolute path; the prompt points at it.

`git pull` before each run. A stale clone runs an out-of-date definition and you won't notice until the output is wrong.

4. Turn on the connectors

- **Linear:** the specialist reads the story and writes its report through it. Without it the run can't report anything.
- **Source control:** an authenticated `gh`, or the GitHub connector, so it can open the PR.

Git itself runs locally in your checkout.

5. Approve tool calls as they come

Claude will ask before running `git`, the test suite, or `gh pr create`. Approve them one at a time rather than pre-approving in `.claude/settings.json`. For the first few stories you want to see what it actually does.

Every story

- ☐ Story read, and it makes sense to you
- ☐ Your questions asked and answered **in the story's comment thread**
- ☐ Story left in To-Do (not moved to `In Progress`)
- ☐ Branch chain set up in the target surface's repo, story branch checked out
- ☐ `git pull` in the framework clone
- ☐ Claude Code session pointed at the workspace folder

Read the story

You're going to review whatever comes back. If it doesn't make sense now, it won't make more sense as a diff.

Ask your questions in the story's comment thread

Not in Slack, not in a call. The specialist reads that thread as part of the story, so anything the architect answers there, it picks up. Anything answered anywhere else, it never sees.

Ask before you dispatch, not during review.

Set the branch chain up

```
main
├── <BRD branch>           one per project
│   └── <epic branch>       the tracker's branch name on the epic
│       └── <story branch>  the tracker's branch name on the story
```

Use the branch names the tracker already assigns to the epic and story; don't make up your own. The architect creates the BRD branch and the epic branch; epic branches come off the BRD branch, never off `main`. You create the story branch off the epic branch and check it out before you dispatch.

The specialist checks this chain and stops if it's wrong. It won't create or re-parent a branch for you.

If an earlier attempt at this story was abandoned, delete its story branch first and cut it again from the epic branch. Closing a PR does not delete its branch, and a re-run on top of the old commits builds on work that was thrown away. The app refuses to dispatch onto such a branch for the same reason.

A cross-story test story is cut after the stories it tests. Create its branch once those have merged into the epic branch: not after every story in the epic, just the ones its coverage needs. Cut it early and the code it exercises isn't underneath it yet.

The prompt

There is one specialist definition. The story's `surface:` label(s) say where it works, not which file to load. Look up each surface's record in the project's `Surfaces` document (and the epic's `Surfaces (override)` document, if it has one) for the repo, the directory it occupies, and any mandatory skills.

Read these files now. They define your role and the contracts you work to:

- `<FRAMEWORK_PATH>/agents/specialist.md`
- `<FRAMEWORK_PATH>/skills/story-contract/SKILL.md`
- `<FRAMEWORK_PATH>/skills/epic-writing/SKILL.md`
- <any mandatory skill the surface registry lists for this surface>

You are the Specialist those files describe. Follow that definition; this message only tells you which story and where.

Assignment: story `<STORY_ID>` – "`<STORY_TITLE>`", under epic `<EPIC_ID>`.

Your story carries the label(s) `<SURFACE_LABELS>`. Your target repo is `./<SURFACE_REPO>`. `<SURFACE_DIRECTORIES>` Every write you make goes there and nowhere else. It is checked out on `<STORY_BRANCH>`; the epic branch is `<EPIC_BRANCH>`. Both names come from the tracker, and I set the chain up before dispatching you. Verify it, do not repair it.

The other repositories in this workspace are sibling surfaces. Read them when you need to confirm what a dependency actually implements rather than what the story claims it does. Do not modify them and do not open a PR against them.

Using the Linear connector, read `<STORY_ID>`'s description and its full comment

thread, then walk up to <EPIC_ID> for the parent epic, its resolved API map, and the linked design issue. The comment thread on <STORY_ID> may carry a question-and-answer exchange between me and the architect from before you were engaged. Read it as part of the story, not as commentary on it.

Then act per your definition: check blocking dependencies, verify the branch chain, read the codebase and its conventions spec, do the story's work, open the PR into <EPIC_BRANCH>, and post your completion report on <STORY_ID>.

If the branch chain is wrong, a blocking dependency is unmerged, or the story has a gap you cannot resolve from the thread, stop and report it rather than deciding for yourself. A blocker you surface is the useful output of this run.

Placeholder	Where it comes from
<FRAMEWORK_PATH>	Absolute path to your intent-to-production clone
<STORY_ID>, <STORY_TITLE>	The story
<EPIC_ID>	The story's parent epic
<SURFACE_LABELS>	The story's surface : labels, e.g. surface:web surface:e2e
<SURFACE_REPO>	Folder name of the target repo in your workspace
<SURFACE_DIRECTORIES>	From the registry: "The surface is the whole repository." or, when a surface has a path , "Within the repository, web lives under web/; e2e lives under e2e/. Write only inside those directories."
<STORY_BRANCH>, <EPIC_BRANCH>	The branch names on the story and the epic

What comes back

A PR from the story branch into the epic branch, titled <story id>: <title>, and a comment on the story reporting one of three outcomes. No label; the comment is the only record:

Outcome	Meaning
Complete	Done. Every acceptance criterion traced or its gap reported, tests pass, PR is open.
Waiting	A story it depends on isn't merged yet. Nothing was written.
Blocked	It hit something it wouldn't guess at: a missing conventions spec, a gap in the story, a wrong branch base, a defect below the epic branch. Read the comment; it names the specific thing.

The PR body carries the acceptance-criteria trace as a task list: one unticked checkbox per criterion, each naming the code that implements it and the test that asserts it, with any gap marked. The reviewer reads the diff against that list and ticks what they verified. The specialist never ticks a box.

CI runs the tests on the PR. A developer other than you reads the diff and merges it into the epic branch. No agent reviews it.

Read the "Questions and assumptions" section of the report. Each entry is a place the story was vague enough that the agent had to decide something. Those are worth passing back to whoever wrote it.

Answering a review

If the reviewer requests changes, you can run the specialist again at the same branch and PR instead of making the changes yourself. Tell it this is a revision round, give it the PR number and the review, and point it at the "A revision round" section of [agents/specialist.md](#). It reads the review, its own report and the PR's trace before touching anything, replies in the thread each comment was left in, and never merges, closes, resolves a conversation, or moves a checkbox. Keep a revision small: feedback too big for one short run was a story change, not a review comment.